Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO



2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



15

Upload de arquivos

Aula 57 a 60 - Desenvolvimento de Sistemas
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI



15.1

Estrutura de pastas

Definindo repositório dos arquivos de upload

Preparando o armazenamento

Criando a pasta das capas

Antes de criar a rota de upload, precisamos definir onde os arquivos enviados serão armazenados.

Na pasta principal do projeto, no mesmo nível do arquivo pom.xml, crie a pasta:

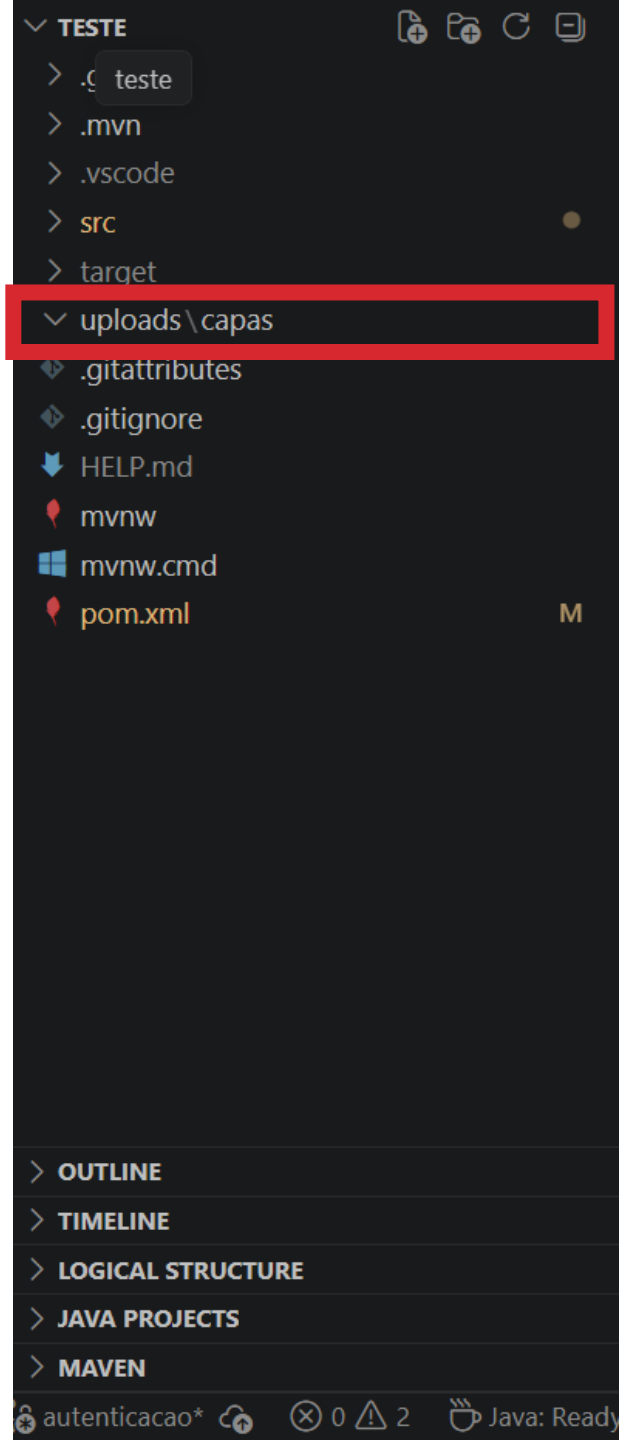
- uploads

Dentro dela, crie:

- capas

Por que ela fica fora de src?

A pasta src guarda os arquivos que fazem parte do código-fonte da aplicação.As capas serão adicionadas durante a execução do sistema. Por isso, devem ficar em uma pasta externa ao código-fonte.



Local de armazenamento

Atenção



A pasta está dentro do projeto apenas para simplificar esta primeira prática.

Em uma aplicação publicada, os arquivos podem ser armazenados fora do projeto ou em serviços próprios de armazenamento.

O que ficará no banco?

Arquivos são armazenados no banco de dados?

Saiba Mais!



O banco de dados não armazenará a imagem completa. Ele guardará somente o nome utilizado para localizar o arquivo:

capa-java.jpg

Enquanto o arquivo estará salvo em: uploads/capas/capa-java.jpg



15.2

Ajustando model

Adicionando coluna que irá armazenar nome do arquivo

Firjan SENAI

Capa no model Livro

Armazenando o nome do arquivo

A imagem será salva na pasta uploads/capas. Porém, o sistema precisa saber qual arquivo pertence a cada livro.

No arquivo Livro.java, adicione:

- Atributo nomeArquivoCapa
- getter e o setter

O que será armazenado?

O atributo não receberá a imagem completa. Ele guardará somente o nome do arquivo:

- 550e8400-capa-java.jpg

Com esse nome, a aplicação conseguirá localizar a imagem em: uploads/capas/550e8400-capa-java.jpg

```java
@Entity
@Table(name = "livro")
public class Livro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @NotBlank(message = "O título é obrigatório")
    private String titulo;

    @NotBlank(message = "O autor é obrigatório")
    private String autor;

    @Min(value = 1, message = "O ano deve ser maior que zero")
    private int anoPublicacao;

    private String nomeArquivoCapa;

    public String getNomeArquivoCapa() {
        return nomeArquivoCapa;
    }

    public void setNomeArquivoCapa(String nomeArquivoCapa) {
        this.nomeArquivoCapa = nomeArquivoCapa;
    }
```



15.3

Adicionando método no service

Criando a lógica para o upload de arquivos

Firjan SENAI

Salvando a capa do livro

Adicionando o método no LivroService

O próximo passo será receber a imagem e salvá-la na pasta uploads/capas.

Não criaremos uma nova classe. Adicionaremos o método diretamente no LivroService.java:

- Adicione as importações
- Adicione o método salvarCapa

Método dentro da classe LivroService

```java
public Livro salvarCapa(Integer id, MultipartFile arquivo) {

    Optional<Livro> livroEncontrado = livroRepository.findById(id);

    if (livroEncontrado.isEmpty()) {
        throw new ResponseStatusException(
                HttpStatus.NOT_FOUND,
                "Livro não encontrado");
    }

    Livro livro = livroEncontrado.get();

    String nomeArquivo = System.currentTimeMillis()
            + "-"
            + arquivo.getOriginalFilename();

    File pasta = new File("uploads/capas");

    if (!pasta.exists()) {
        pasta.mkdirs();
    }

    File destino = new File(
            pasta,
            nomeArquivo).getAbsoluteFile();

    try {
        arquivo.transferTo(destino);
    } catch (IOException erro) {
        erro.printStackTrace();

        throw new ResponseStatusException(
                HttpStatus.INTERNAL_SERVER_ERROR,
                "Não foi possível salvar a capa");
    }

    livro.setNomeArquivoCapa(nomeArquivo);

    return livroRepository.save(livro);
}
```

O que é MultipartFile?

MultipartFile arquivo

Saiba Mais!



É o tipo utilizado pelo Spring para representar um arquivo recebido em uma requisição.

Assim como @RequestBody recebe os dados de um JSON, o MultipartFile receberá a imagem selecionada no Postman.

Fique ON



Evitando nomes repetidos

Antes do nome original, adicionamos o horário atual: System.currentTimeMillis()

Assim, um arquivo chamado capa.jpg poderá ser salvo como: 1792027550123-capa.jpg

Isso reduz a possibilidade de uma nova imagem substituir outra que possua o mesmo nome.



15.4

Criando rota no controller

Disponibilizando o endpoint /livros/id/capa para upload de imagens

Rota para enviar a capa

Recebendo o arquivo no controller

No arquivo LivroController.java:

- Adicione as importações
- Adicione o método enviarCapa

A rota criada será:

- POST /livros/{id}/capa

O arquivo enviado será recebido neste parâmetro:

- @RequestParam("arquivo") MultipartFile arquivo

Adicione esse método dentro da classe LivroController

```
@PostMapping("/{id}/capa")
public ResponseEntity<Livro> enviarCapa(
        @PathVariable Integer id,
        @RequestParam("arquivo") MultipartFile arquivo) {

    Livro livroAtualizado = livroService.salvarCapa(id, arquivo);

    return ResponseEntity.ok(livroAtualizado);
}
```



15.5

Testando rota

Upload de arquivo no postman usando o form-data

Testando o upload da capa

Enviando uma imagem pelo Postman

Primeiro, confirme o ID do livro que receberá a capa:

- GET http://localhost:8080/livros

Depois, crie uma nova requisição:

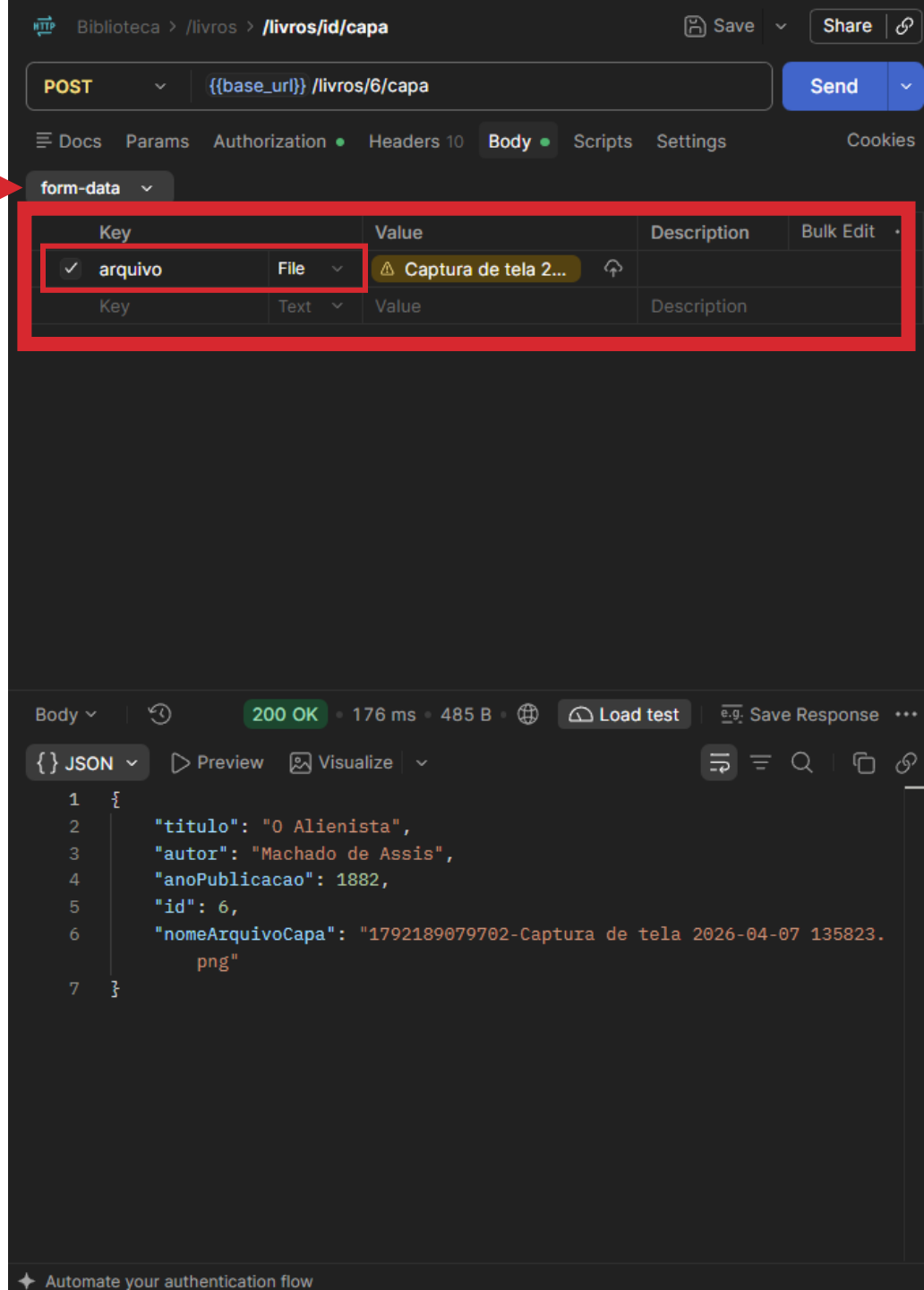
- POST http://localhost:8080/livros/1/capa

Substitua o número 1 pelo ID do livro escolhido. Lembre também de configurar a autenticação

Selecionando a imagem

Na guia Body:

- selecione form-data;
- crie uma key chamada arquivo;
- altere o tipo de Text para File;
- selecione uma imagem do computador;
- clique em Send.



Adicionando uploads no .gitignore

Atenção



A pasta uploads armazenará imagens enviadas durante os testes e não deve ser enviada para o GitHub.

No arquivo .gitignore, adicione:
uploads/

Assim, o Git ignorará a pasta e todos os arquivos existentes dentro dela.

Saiba Mais!



Isso evita:

- enviar imagens de teste para o repositório;
- aumentar desnecessariamente o tamanho do projeto;
- compartilhar arquivos enviados pelos usuários.

A aplicação continuará funcionando, pois o método mkdirs() criará a pasta automaticamente quando necessário.

Foto de identificação do aluno

Até aqui, a API já permite enviar uma imagem e relacioná-la à capa de um livro. Agora, utilize o mesmo processo para adicionar uma foto 3x4 ao cadastro de um aluno e reforçar o funcionamento do upload de arquivos.

Desafio



Crie a pasta uploads/fotos-alunos e adicione ao model Aluno o atributo nomeArquivoFoto.
Implemente o salvamento da imagem no AlunoService e crie a rota POST /alunos/{id}/foto no AlunoController.

Teste no Postman utilizando Bearer Token e Body → form-data, com uma chave do tipo File chamada arquivo.
Confirme o arquivo na pasta e o nome salvo no banco.

Firjan SENAI